

Machine Learning

CMPSCI 589

Bruno C. da Silva

bsilva@cs.umass.edu

Evaluation Metrics (2/2)



Review: Classification Performance

- The performance of a classifier can be visualized as a **confusion matrix**
- Suppose a binary classification problem (two classes only: c_1 and c_2)

Predicted class

┌───────────┐

	c_1	c_2
c_1	11	1
c_2	3	9

True class ┌───────────┐

Detailed description: The confusion matrix is a 2x2 grid. The columns represent the predicted classes, labeled c_1 and c_2 in blue boxes at the top. The rows represent the true classes, labeled c_1 and c_2 in green boxes on the left. The diagonal elements (11 and 9) represent correct classifications. The off-diagonal elements (1 and 3) represent misclassifications. A bracket above the columns is labeled 'Predicted class' and a bracket to the left of the rows is labeled 'True class'.

Review: Classification Performance

- The performance of a classifier can be visualized as a **confusion matrix**
- Suppose a binary classification problem (two classes only: c_1 and c_2)

Predicted class

┌───────────┐

		+	-
True class	+	11	1
	-	3	9

Review: Classification Performance

- The performance of a classifier can be visualized as a **confusion matrix**
- Suppose a binary classification problem (two classes only: c_1 and c_2)

		Predicted class	
		+	-
True class	+	11	1
	-	3	9

Quantities of interest:

True Positive (**TP**)

True Negative (**TN**)

False Positive (**FP**)

False Negative (**FN**)

Review: Classification Performance

- The performance of a classifier can be visualized as a **confusion matrix**
- Suppose a binary classification problem (two classes only: c_1 and c_2)

		Predicted class	
		+	-
True class	+	11	1
	-	3	9

Quantities of interest:

True Positive (**TP**)

True Negative (**TN**)

False Positive (**FP**)

False Negative (**FN**)

Review: Classification Performance

- The performance of a classifier can be visualized as a **confusion matrix**
- Suppose a binary classification problem (two classes only: c_1 and c_2)

		Predicted class	
		+	-
True class	+	11	1
	-	3	9

Quantities of interest:

True Positive (**TP**)

True Negative (**TN**)

False Positive (**FP**)

False Negative (**FN**)

Review: Classification Performance

- The performance of a classifier can be visualized as a **confusion matrix**
- Suppose a binary classification problem (two classes only: c_1 and c_2)

		Predicted class	
		+	-
True class	+	11	1
	-	3	9

Quantities of interest:

True Positive (**TP**)

True Negative (**TN**)

False Positive (**FP**)

False Negative (**FN**)

Review: Classification Performance

- The performance of a classifier can be visualized as a **confusion matrix**
- Suppose a binary classification problem (two classes only: c_1 and c_2)

		Predicted class	
		+	-
True class	+	11	1
	-	3	9

Quantities of interest:

True Positive (**TP**)

True Negative (**TN**)

False Positive (**FP**)

False Negative (**FN**)

Limitations of Accuracy

Predicted class



	+	-
+	10	2
-	4	8

Model 1

Predicted class



	+	-
+	18	0
-	6	0

Model 2

$$\text{acc}(\text{Model1}) = \frac{18}{24} = 0.75$$

$$\text{acc}(\text{Model2}) = \frac{18}{24} = 0.75$$

Same accuracy
Are they equally good?



**Model 2 incorrectly predicts
all instances of the negative class!**

Limitations of Accuracy

- In some problems, we may be interested in minimizing the number of False Negatives
 - e.g., when diagnosing cancer
 - don't want to say “No cancer, you're fine!”
 - even though the person might have cancer...
- In other problems, we may be interested in minimizing the number of False Positives
 - e.g., loan concession
 - don't want to say “Loan approved, here's a million dollars!”
 - even though the person most likely will not pay back...

Precision

True class

Predicted class

	+	-
+	TP	FN
-	FP	TN

$$\text{precision}(f) = \frac{TP}{TP + FP}$$

- If the classifier says “the class is positive!”, how often is it correct?
- Percentage of positive predictions that are correct
- That is:
 - out of all instances predicted to be of the positive class
 - how many indeed belong to the positive class?

Recall

Predicted class

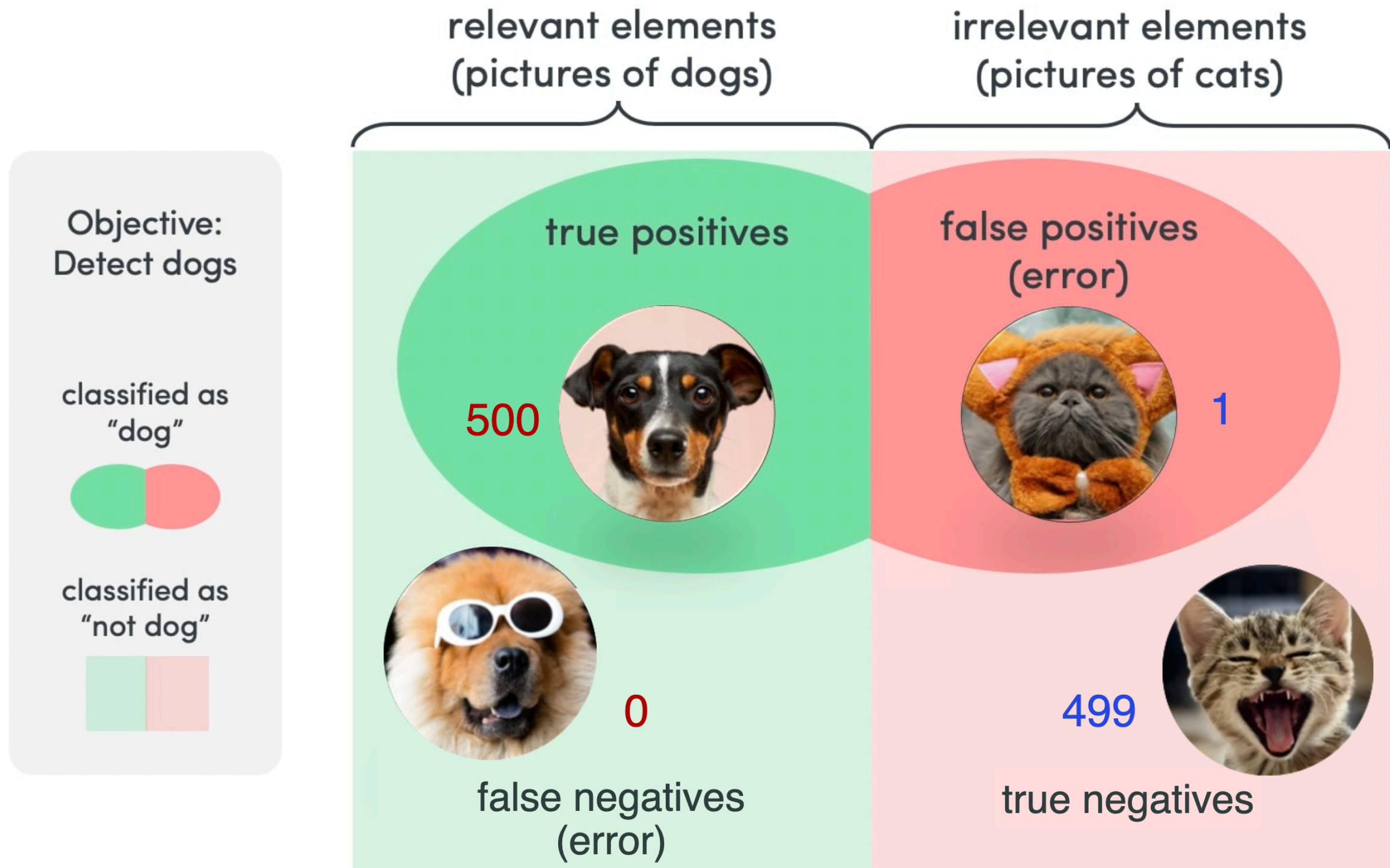
	+	-
+	TP	FN
-	FP	TN

True class

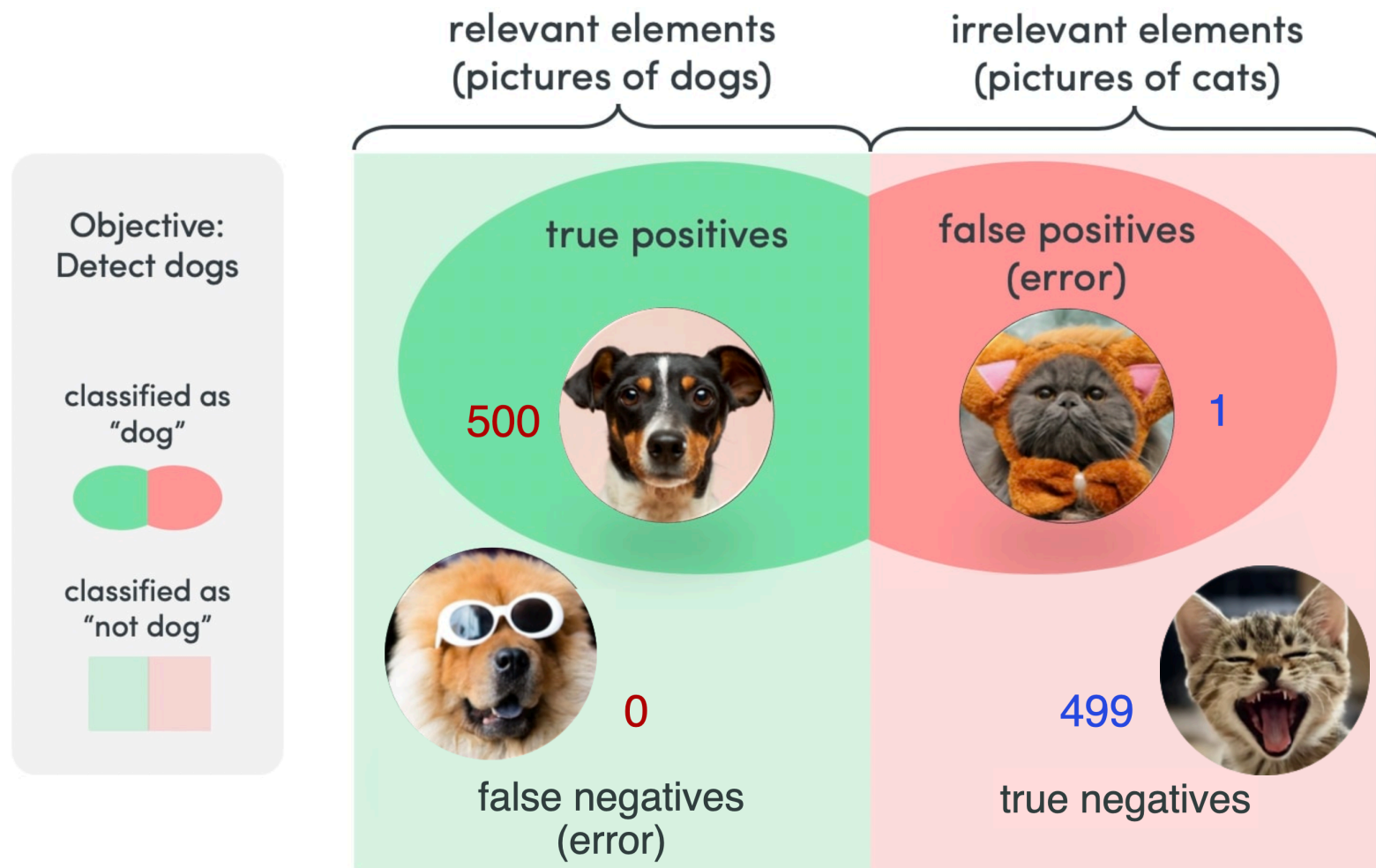
$$\text{recall}(f) = \frac{TP}{TP + FN}$$

- Out of all positive instances...
 - how many does the classifier identify as being positive?
- That is:
 - percentage of correct classifications when considering the positive class

Precision vs Recall



Precision vs Recall



Precision

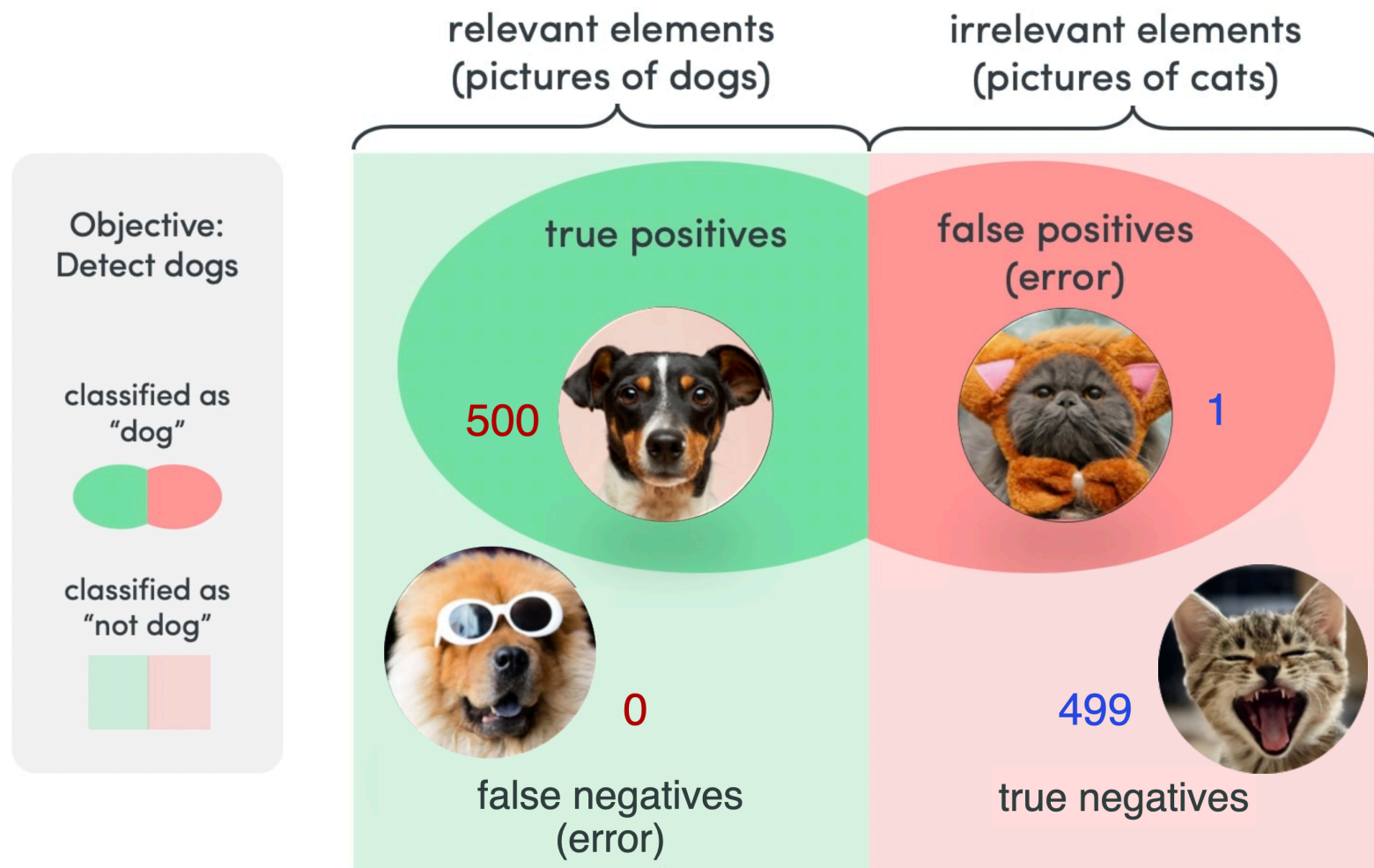
What did the model classify as being a dog?



Out of those, how many were correct?

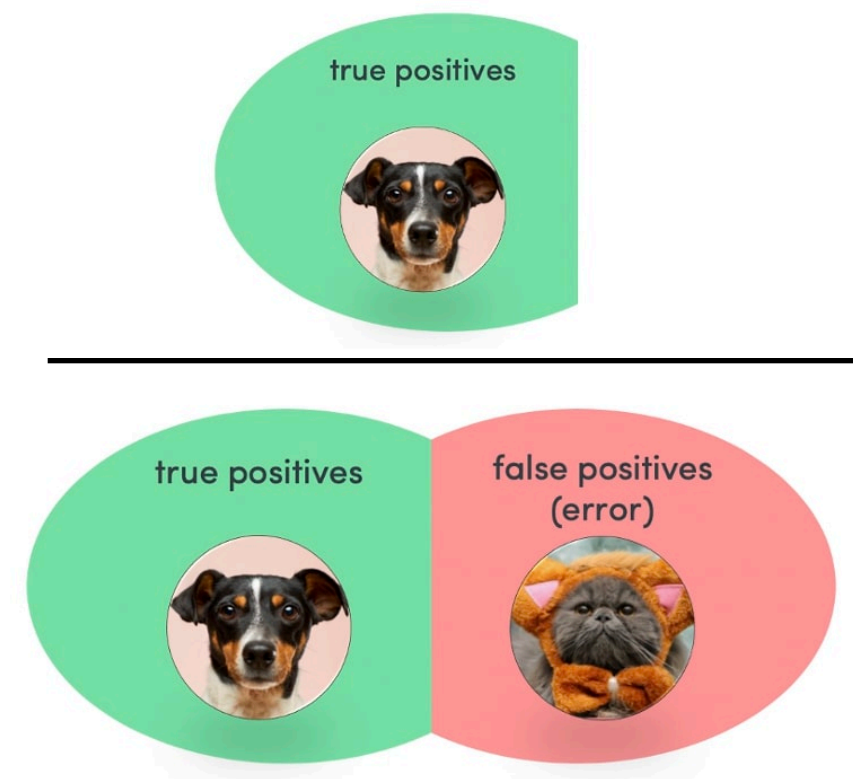


Precision vs Recall



Precision

=



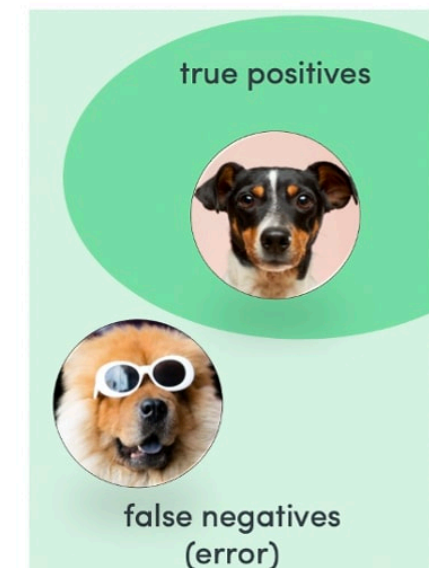
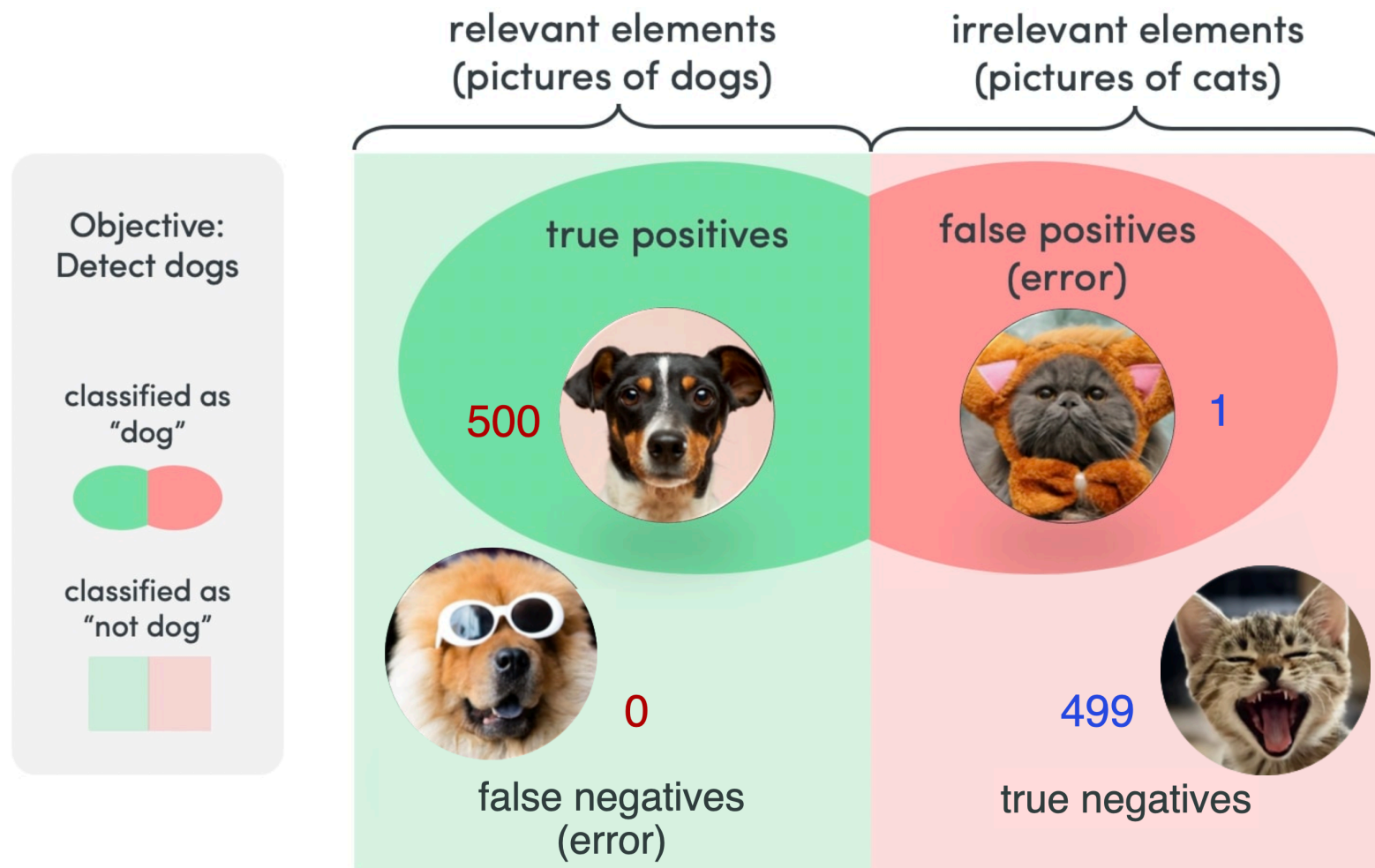
$$\text{precision}(f) = \frac{TP}{TP + FP}$$



Precision vs Recall

Recall

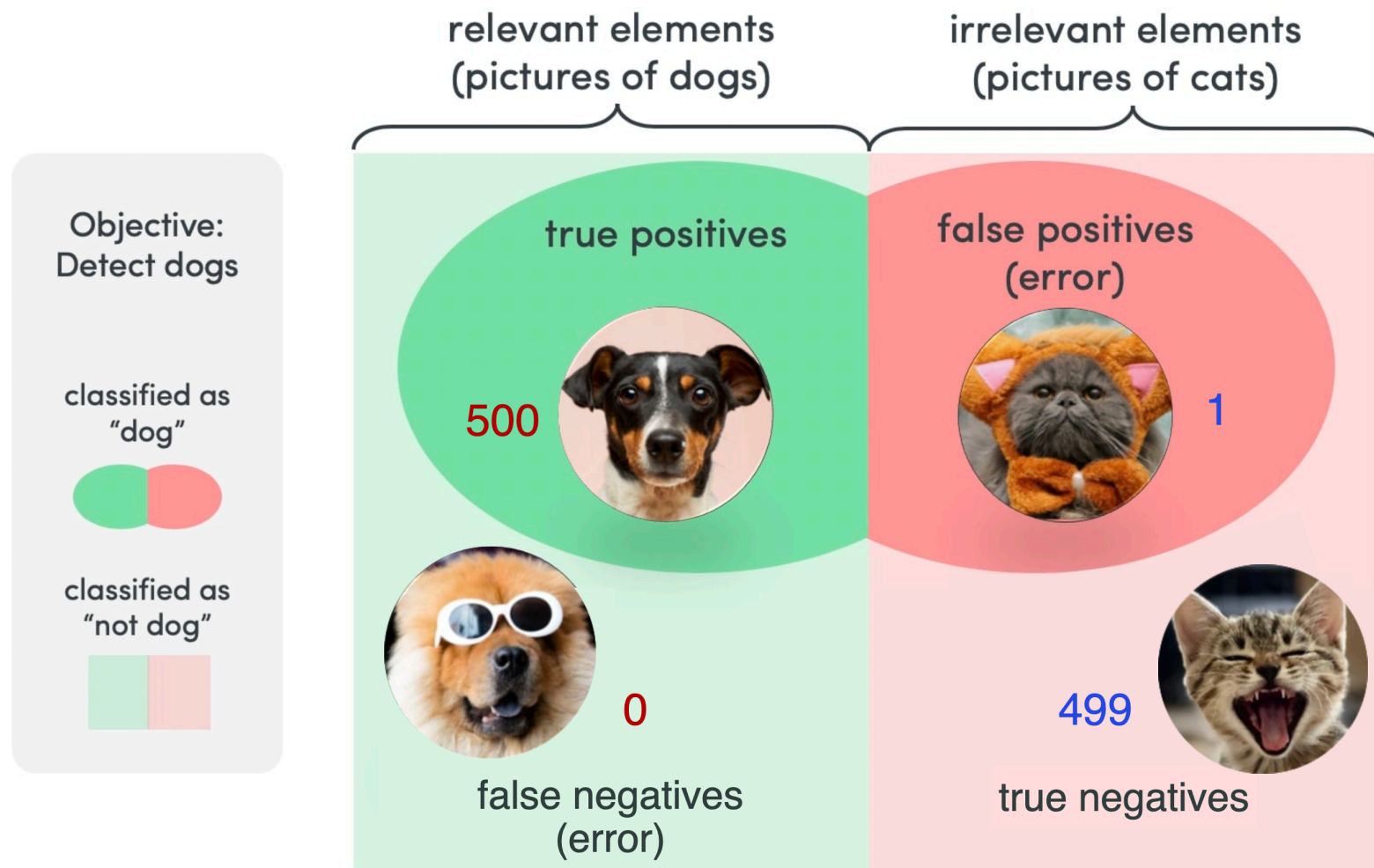
How many dogs are there?



Out of those, how many were correctly identified?

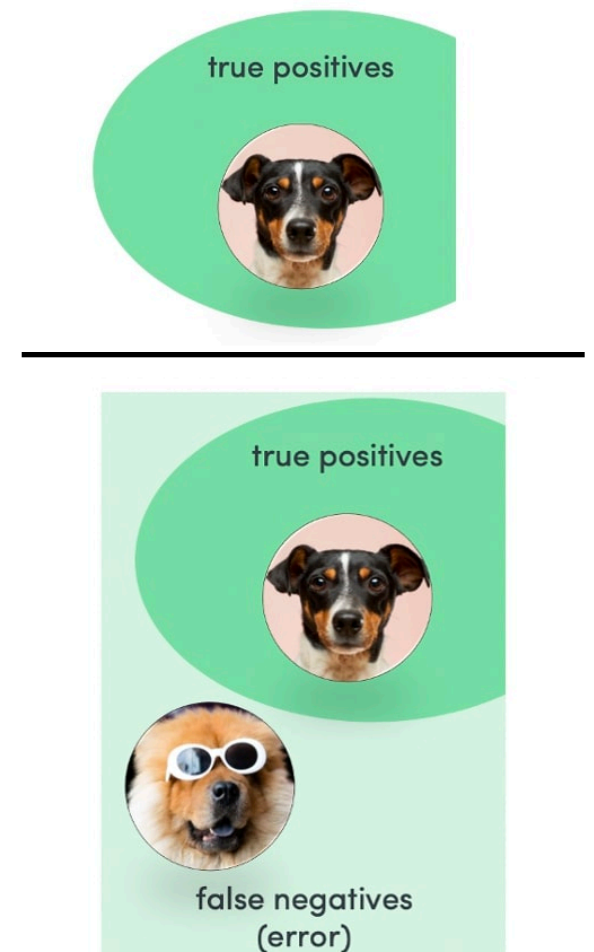


Precision vs Recall



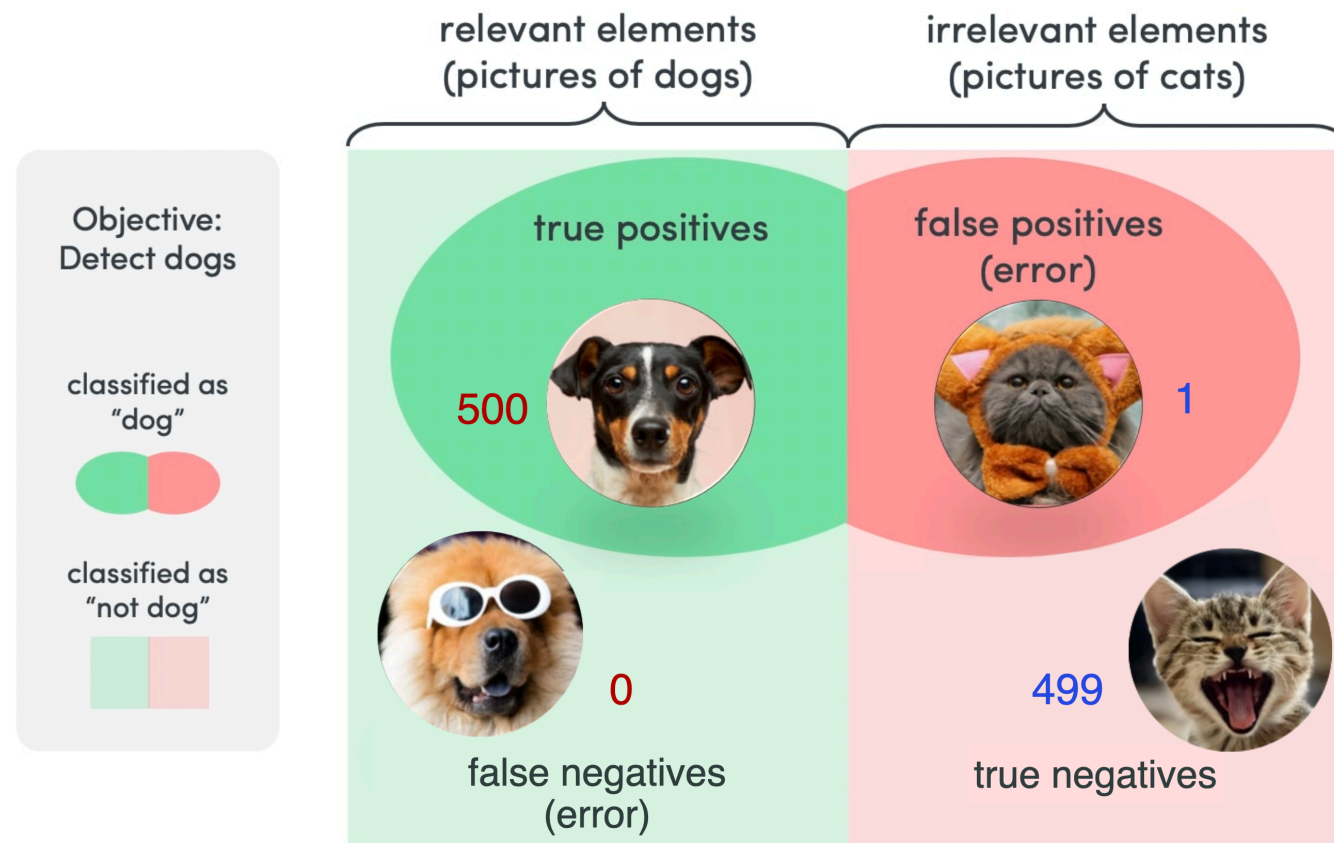
Recall

=



$$\text{recall}(f) = \frac{TP}{TP + FN}$$

Precision vs Recall

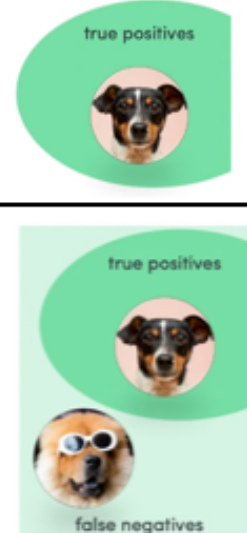


Precision = $\frac{\text{true positives}}{\text{true positives} + \text{false positives (error)}}$



$$= 500/(500+1) = 0.998$$

Recall = $\frac{\text{true positives}}{\text{true positives} + \text{false negatives (error)}}$

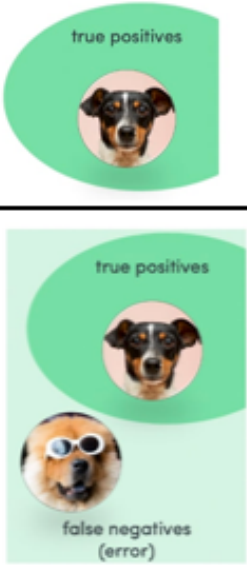


$$= 500/(500+0) = 1$$

Precision vs Recall

Precision =  = $\frac{TP}{TP + FP}$

= $500/(500+1) = 0.998$

Recall =  = $\frac{TP}{TP + FN}$

= $500/(500+0) = 1$

“Whenever we say it’s a dog,
we’re usually right”



Avoiding making wrong detections

“We’re correctly identifying
pretty much all dogs that exist”



Detecting the relevant items/instances

Precision vs Recall

Precision

“Whenever we say it’s a dog,
we’re usually right”



Avoiding making wrong detections

Recall

“We’re correctly identifying
pretty much all dogs that exist”



Detecting the relevant items/instances

- Usually there’s a trade-off between Precision and Recall
 - It’s easy to have 100% recall...
 - Just classify all instances as dogs....
 - But then what happens to the precision?
 - It’s going to misclassify all cats!

Precision vs Recall

Precision

“Whenever we say it’s a dog,
we’re usually right”



Avoiding making wrong detections

Recall

“We’re correctly identifying
pretty much all dogs that exist”



Detecting the relevant items/instances

- Usually there’s a trade-off between Precision and Recall

Quantifying and optimizing this trade-off

→ **F-Score**

F-Score

Quantifying and optimizing this trade-off

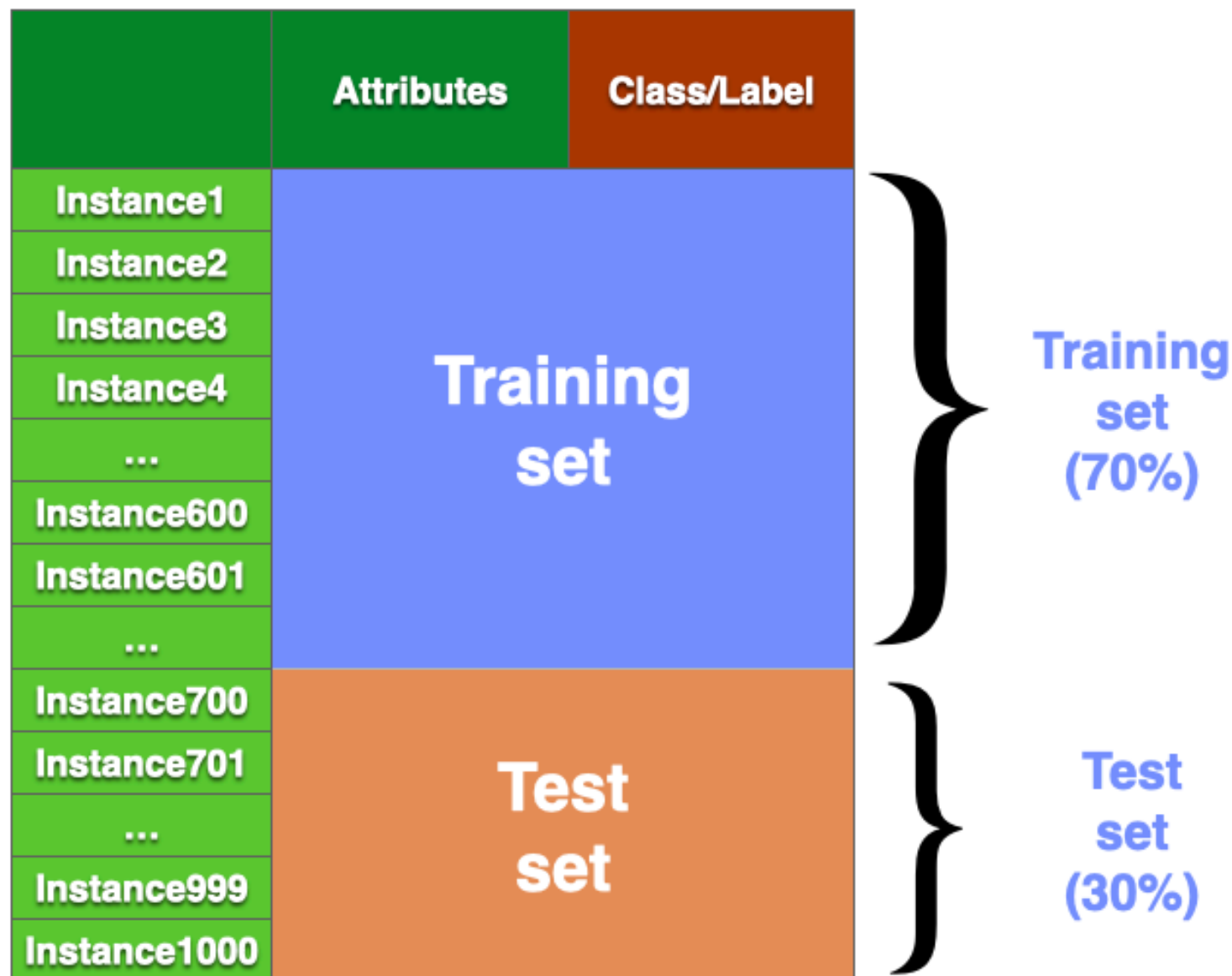
- **F-Score:** weighted harmonic average of precision and recall
 - where we consider that recall is β times more important than precision

$$F_{\beta}(f) = (1 + \beta^2) \frac{\text{precision}(f) \text{ recall}(f)}{(\beta^2 \text{ precision}(f)) + \text{recall}(f)}$$

$$\beta = 1 \rightarrow F_1(f) = 2 \frac{\text{precision}(f) \text{ recall}(f)}{\text{precision}(f) + \text{recall}(f)}$$

Same weight/importance associated both with precision and recall

Review: Holdout Method



Limitation:

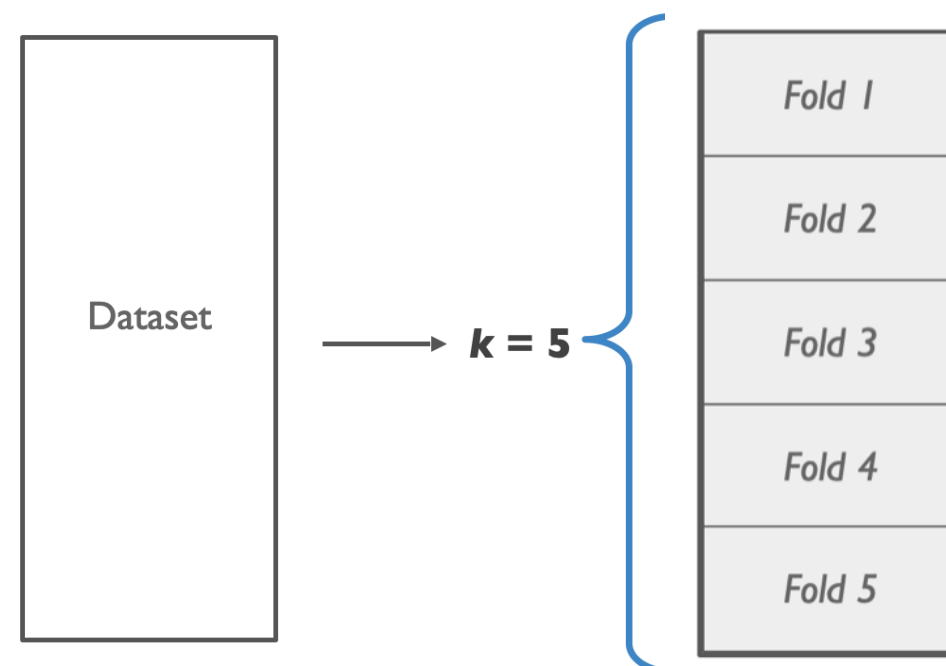
- Evaluates performance on a single test set
- What if we were unlucky when we split?
 - what if test set is particularly “easy”?
 - will **overestimate performance**
 - what if test set is particularly “hard”?
 - will **underestimate performance**

Idea:

- Evaluate performance on multiple test sets
- Increases performance evaluation robustness
- How to generate multiple test sets?

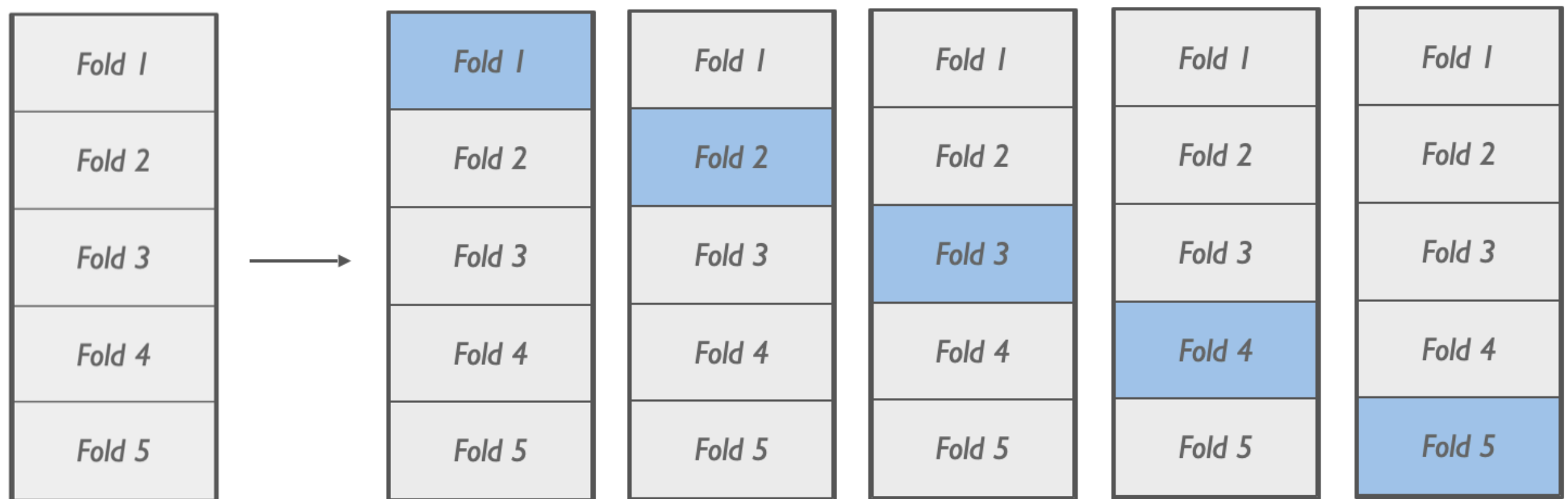
k-fold Cross-Validation

- Splits the original dataset into k partitions (*folds*)
 - disjoint and approximately same size
- Picks $k-1$ folds and use them to train the model
 - The remaining fold is used as the test set
- Repeat this process k times
 - Each time, a different fold is used as the test set
- Model performance → average performance over the k folds used to test the model



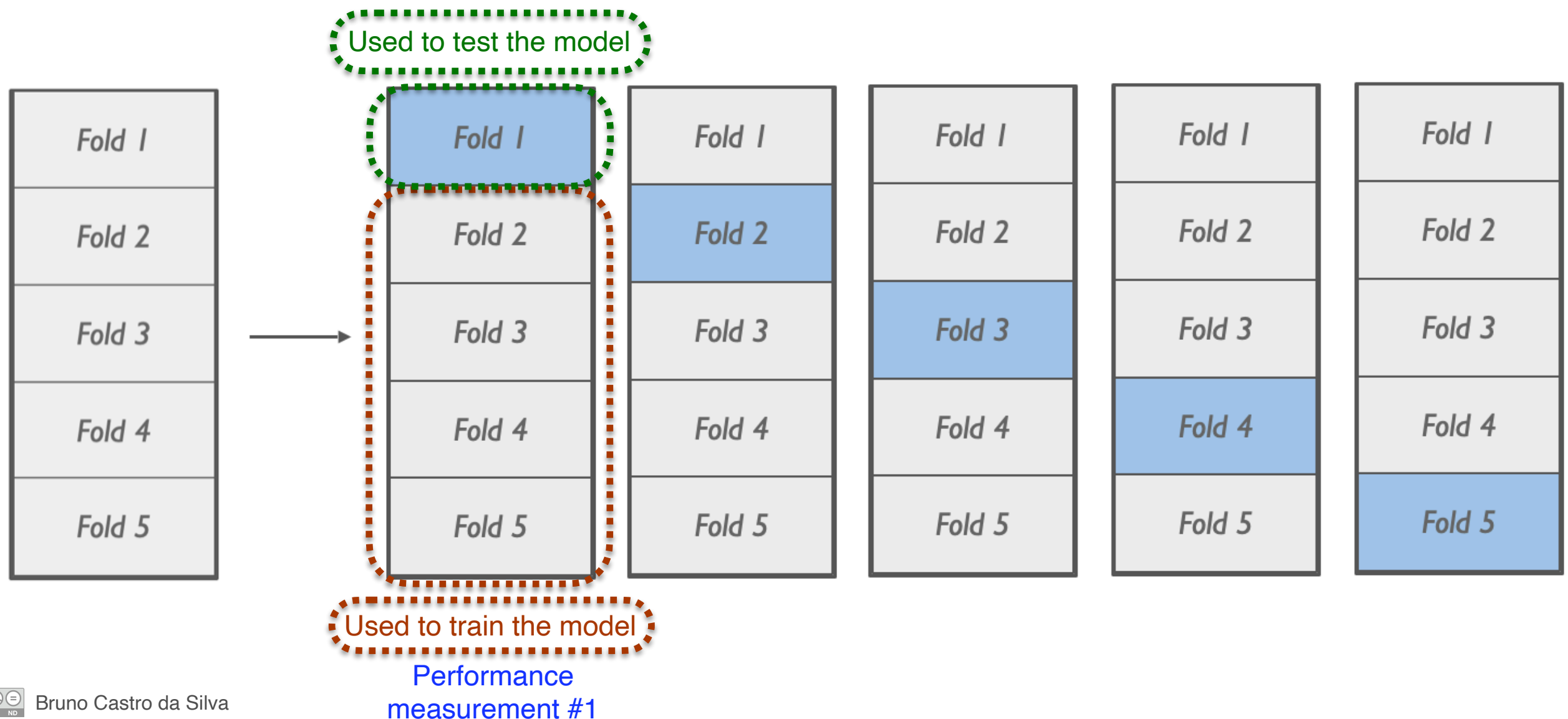
k-fold Cross-Validation

- Picks $k-1$ folds and use them to train the model
 - The remaining fold is used as the test set
- Repeat this process k times
 - Each time, a different fold is used as the test set
- **Model performance** → average performance over the k folds used to test the model



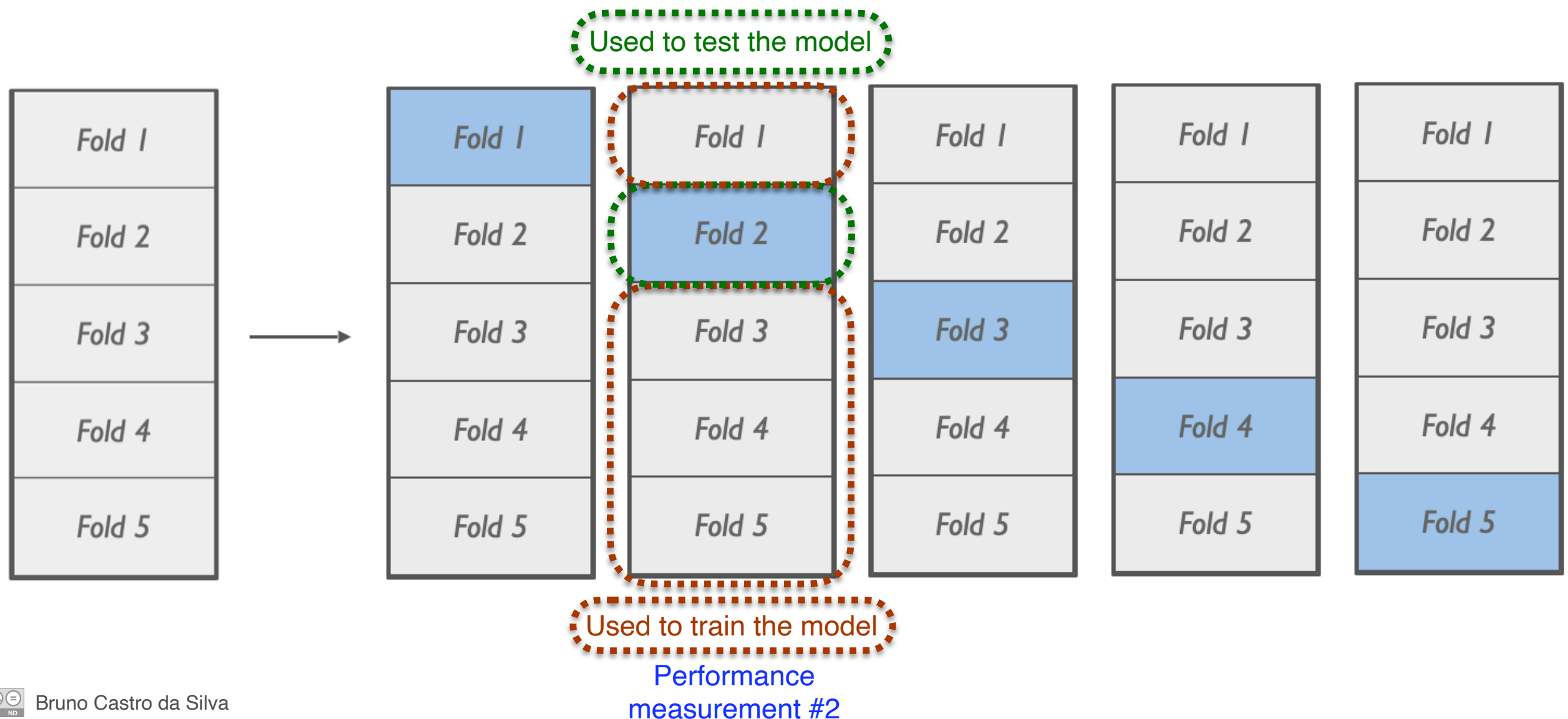
k-fold Cross-Validation

- Picks $k-1$ folds and use them to train the model
 - The remaining fold is used as the test set
- Repeat this process k times
 - Each time, a different fold is used as the test set
- **Model performance** → average performance over the k folds used to test the model



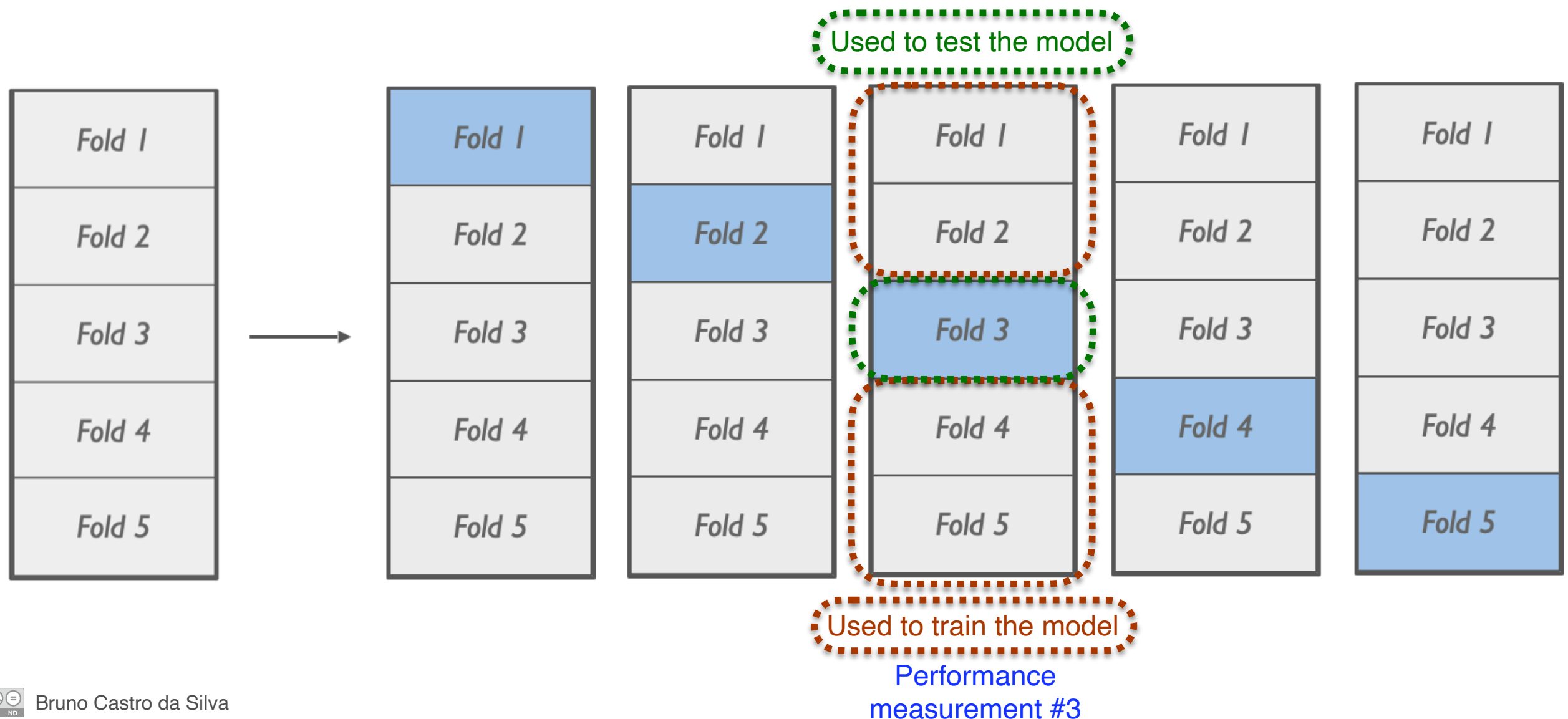
k-fold Cross-Validation

- Picks $k-1$ folds and use them to train the model
 - The remaining fold is used as the test set
- Repeat this process k times
 - Each time, a different fold is used as the test set
- **Model performance** → average performance over the k folds used to test the model



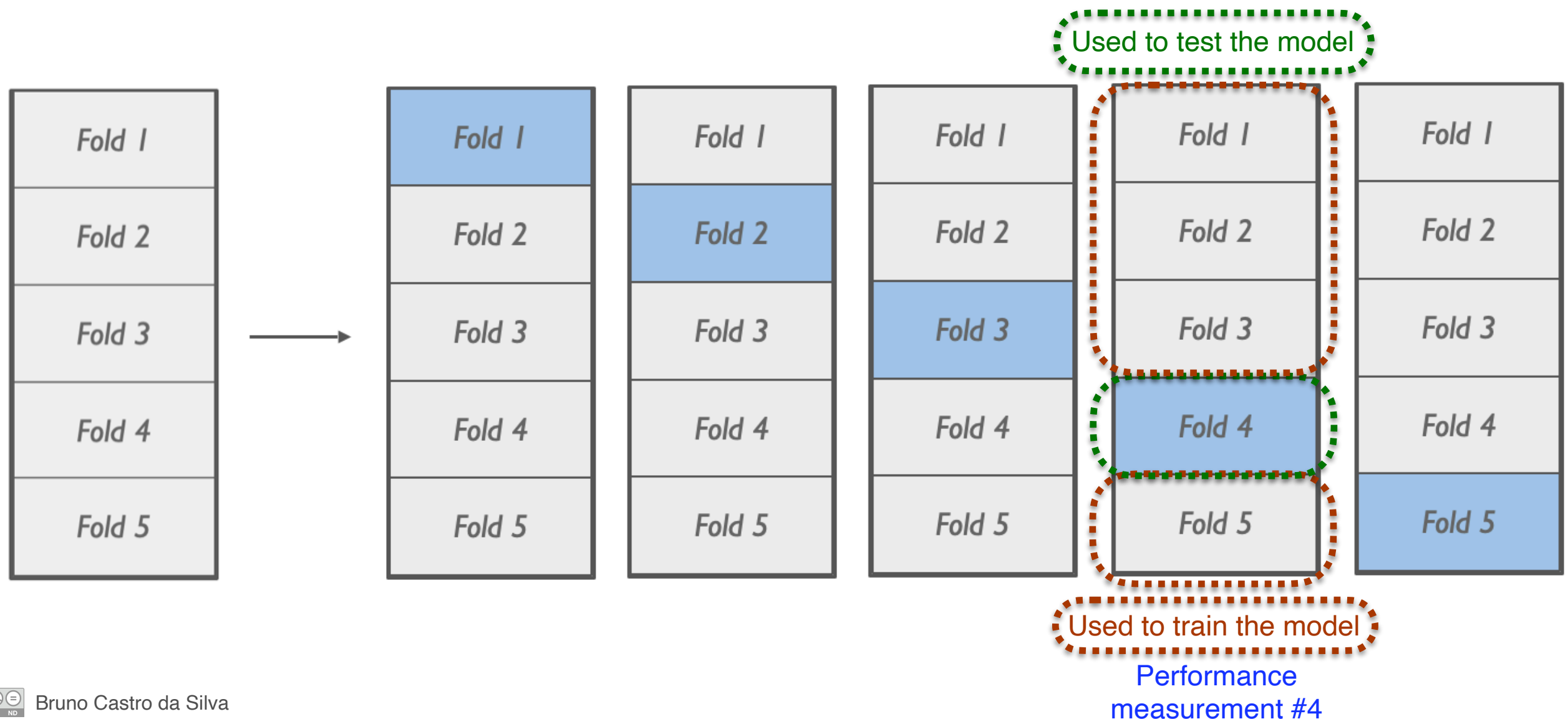
k-fold Cross-Validation

- Picks $k-1$ folds and use them to train the model
 - The remaining fold is used as the test set
- Repeat this process k times
 - Each time, a different fold is used as the test set
- **Model performance** → average performance over the k folds used to test the model



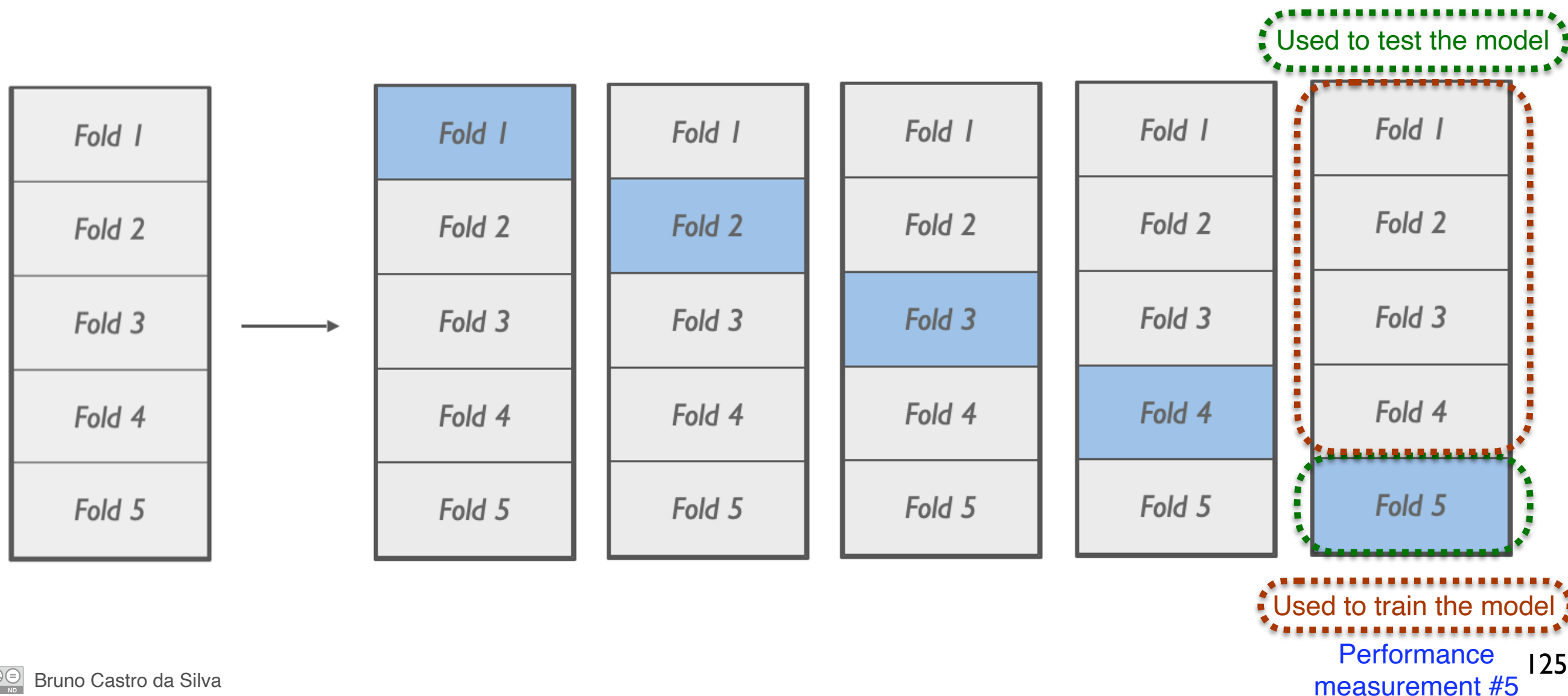
k-fold Cross-Validation

- Picks $k-1$ folds and use them to train the model
 - The remaining fold is used as the test set
- Repeat this process k times
 - Each time, a different fold is used as the test set
- **Model performance** → average performance over the k folds used to test the model



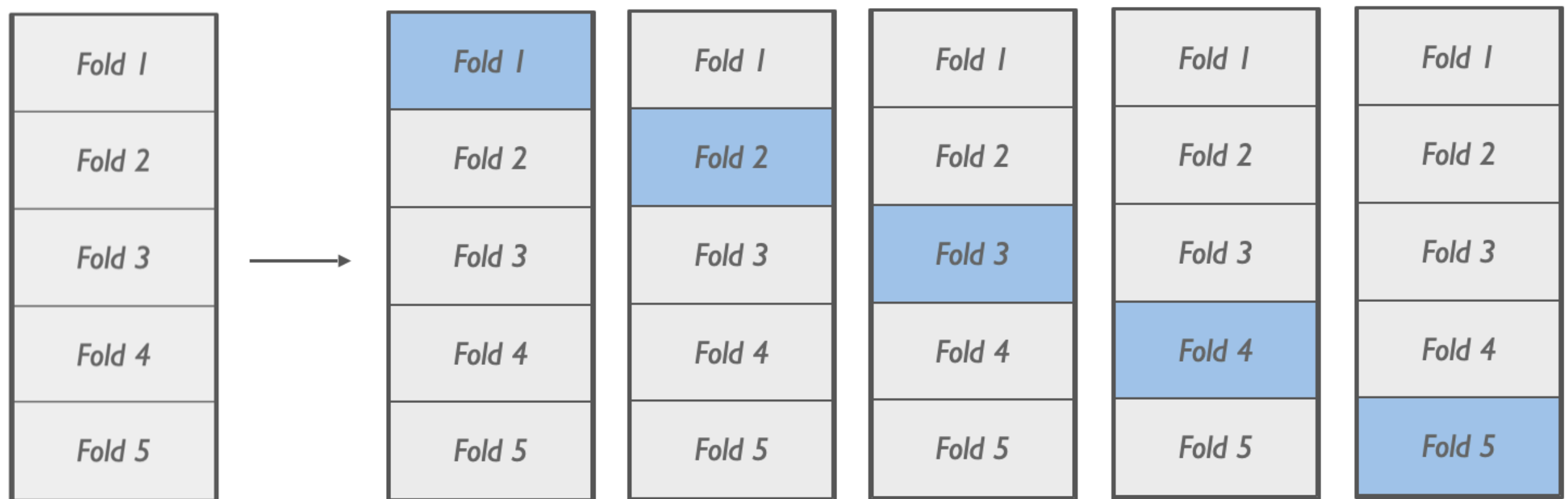
k-fold Cross-Validation

- Picks $k-1$ folds and use them to train the model
 - The remaining fold is used as the test set
- Repeat this process k times
 - Each time, a different fold is used as the test set
- **Model performance** → average performance over the k folds used to test the model



k-fold Cross-Validation

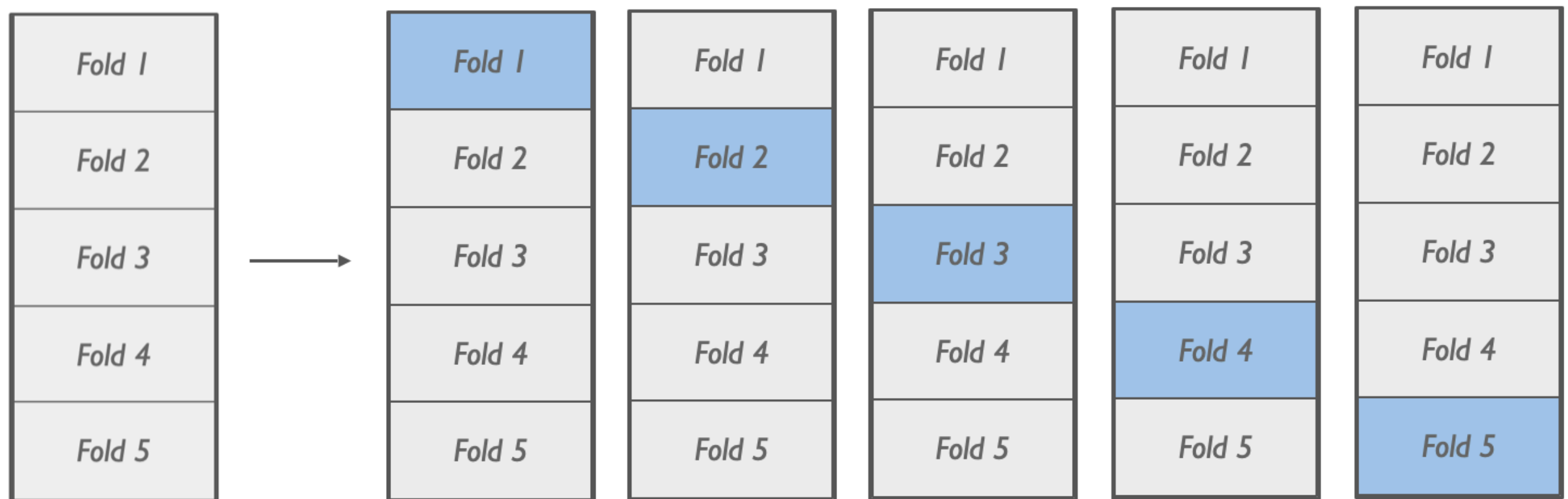
- Picks $k-1$ folds and use them to train the model
 - The remaining fold is used as the test set
- Repeat this process k times
 - Each time, a different fold is used as the test set
- **Model performance** → average performance over the k folds used to test the model



$$\text{Model performance} = (\text{Perf1} + \text{Perf2} + \text{Perf3} + \text{Perf4} + \text{Perf5}) / 5$$

k-fold Cross-Validation

- Picks $k-1$ folds and use them to train the model
 - The remaining fold is used as the test set
- Repeat this process k times
 - Each time, a different fold is used as the test set
- **Model performance** → average performance over the k folds used to test the model



Typically, $k=5$ or $k=10$

Model Selection

How do we compare learning algorithms (or their hyper-parameters) and identify the best?

	Attributes	Class/Label	
Instance1	1.17	A	}
Instance2	2.97	B	
Instance3	3.26	B	
Instance4	4.69	B	
...			
Instance600	6.00	B	}
Instance601	6.41	A	
...			
Instance700	6.00	A	}
Instance701	6.41	A	
...			
Instance999	3.26	A	}
Instance1000	4.69	B	

Training set (70%)

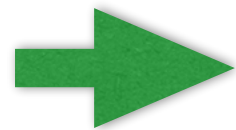
Validation set (15%)

Test set (15%)

Model Selection

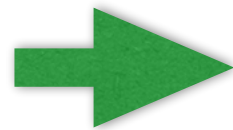
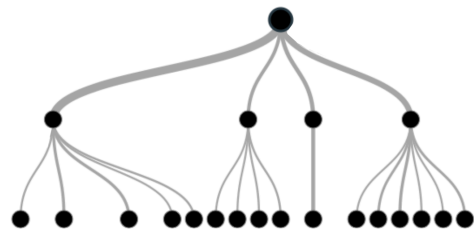
How do we compare learning algorithms (or their hyper-parameters) and identify the best?

(M₁)



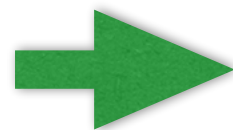
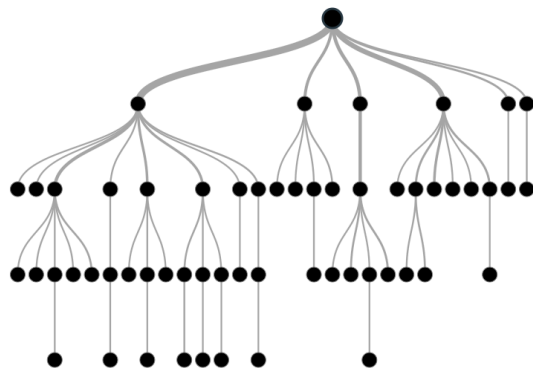
Extremely small/simple decision tree
Probably underfitting

(M₂)



Reasonably-sized decision tree
Possibly the right complexity

(M₃)



Very complex/deep decision tree

Model Selection

How do we compare learning algorithms? Repeat this k times (Cross Validation) Identify the best?

(M₁)



Repeat this k times (Cross Validation)

Obtain average performance of M₁:

CrossValidation(M₁)

Extremely small/simple decision tree

Probably underfitting

Train using the **training set** (70% of data) → M₁
Evaluate on **validation set** (15% of data) → Performance(M₁)

(M₂)



Reasonably-sized decision tree

Possibly the right complexity

(M₃)



Very complex/deep decision tree

Probably overfitting

Model Selection

How do we compare learning algorithms (or their hyper-parameters) and identify the best?

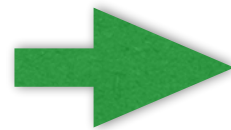
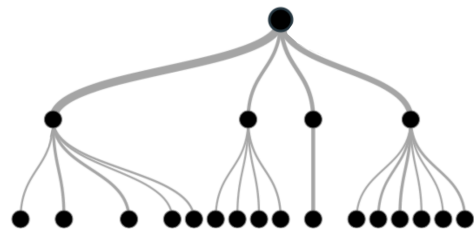
(M₁)



Extremely small/simple decision tree
Probably underfitting

Train using the **training set** (70% of data) → M₁
Evaluate on **validation set** (15% of data) → Performance(M₁)

(M₂)



Reasonably-sized decision tree
Possibly the right complexity

Train using the **training set** (70% of data) → M₂
Evaluate on **validation set** (15% of data) → Performance(M₂)

(M₃)



Very complex/deep decision tree
Probably overfitting

Model Selection

How do we compare learning algorithms (or their hyper-parameters) and identify the best?

(M_1)



Extremely small/simple decision tree
Probably underfitting

Train using the **training set** (70% of data) $\rightarrow M_1$
Evaluate on **validation set** (15% of data) $\rightarrow \text{Performance}(M_1)$

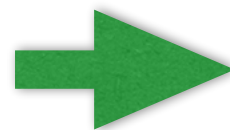
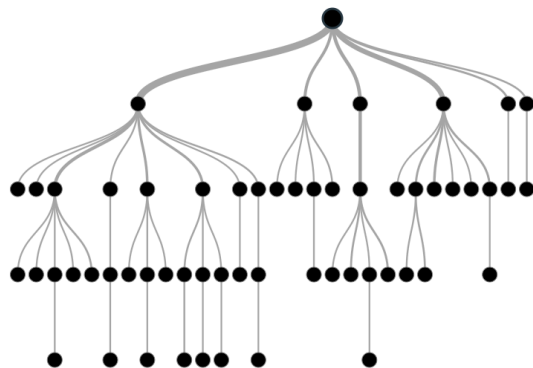
(M_2)



Reasonably-sized decision tree
Possibly the right complexity

Train using the **training set** (70% of data) $\rightarrow M_2$
Evaluate on **validation set** (15% of data) $\rightarrow \text{Performance}(M_2)$

(M_3)



Very complex/deep decision tree
Probably overfitting

Train using the **training set** (70% of data) $\rightarrow M_3$
Evaluate on **validation set** (15% of data) $\rightarrow \text{Performance}(M_3)$

Model Selection

How do we compare learning algorithms (or their hyper-parameters) and identify the best?

(M₁)



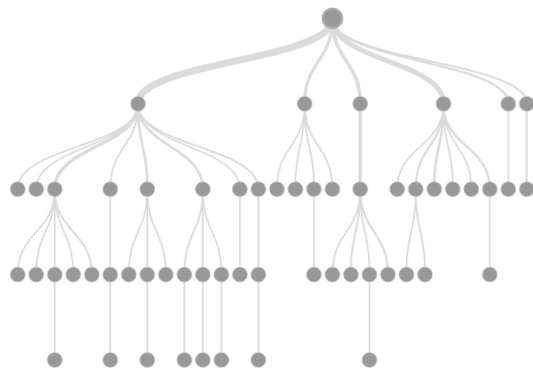
Train using the **training set** (70% of data) → **M₁**
Evaluate on **validation set** (15% of data) → Performance(**M₁**)

(M₂)



Train using the **training set** (70% of data) → **M₂**
Evaluate on **validation set** (15% of data) → Performance(**M₂**)

(M₃)



Train using the **training set** (70% of data) → **M₃**
Evaluate on **validation set** (15% of data) → Performance(**M₃**)

Selects model **M_i (i=1...3)**
with best cross-validation
performance,
CrossValidation(M_i)



Evaluates selected model
on the **test set** (15% of data):
Final Predicted Performance

Model Selection

How do we compare learning algorithms (or their hyper-parameters) and identify the best?

	Attributes	Class/Label	
Instance1	1.17	A	}
Instance2	2.97	B	
Instance3	3.26	B	
Instance4	4.69	B	
...			
Instance600	6.00	B	}
Instance601	6.41	A	
...			
Instance700	6.00	A	}
Instance701	6.41	A	
...			
Instance999	3.26	A	}
Instance1000	4.69	B	

Training set
(70%)

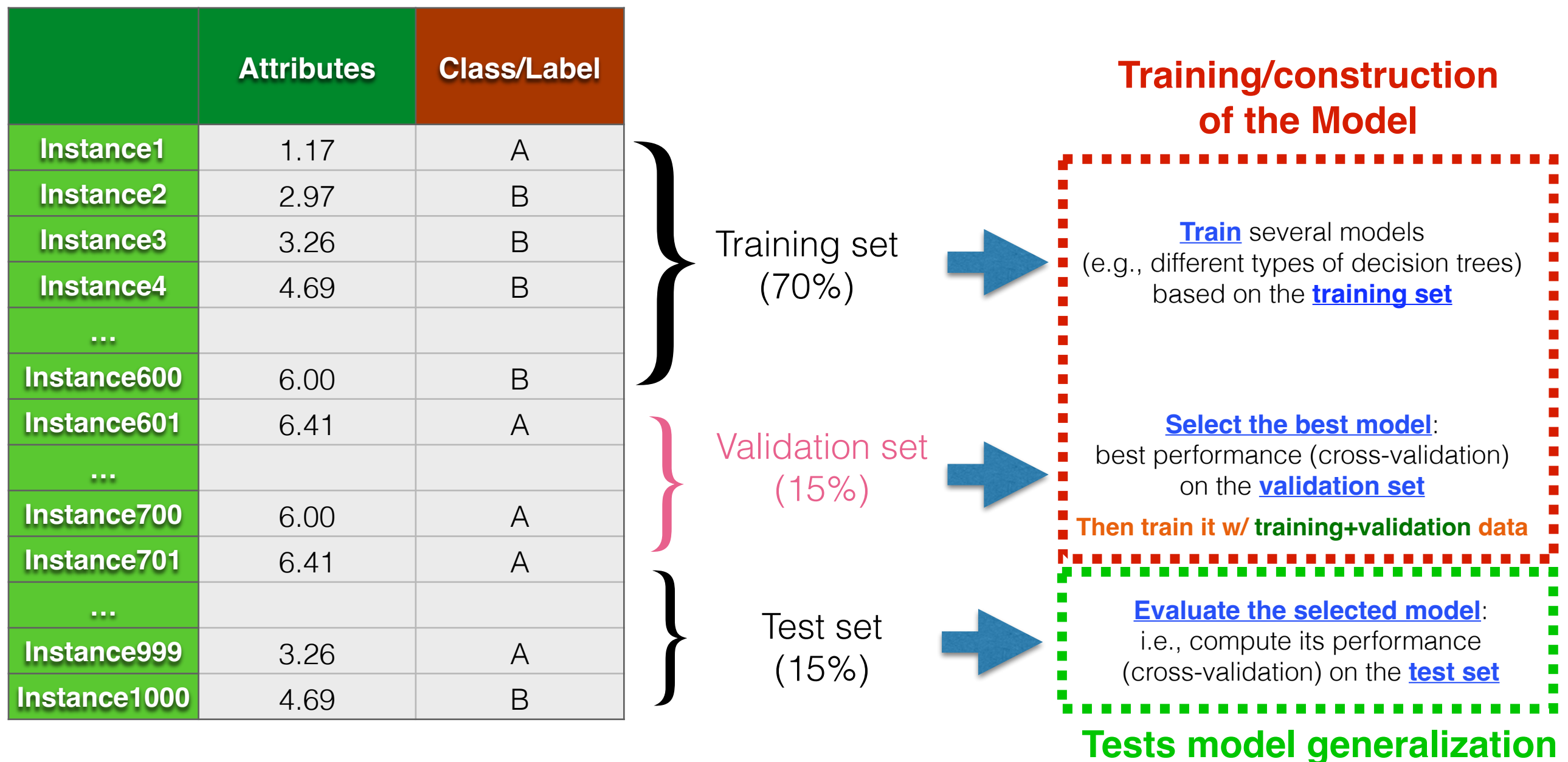
Validation set
(15%)

Test set
(15%)

**Why do we
need three
separate sets?**

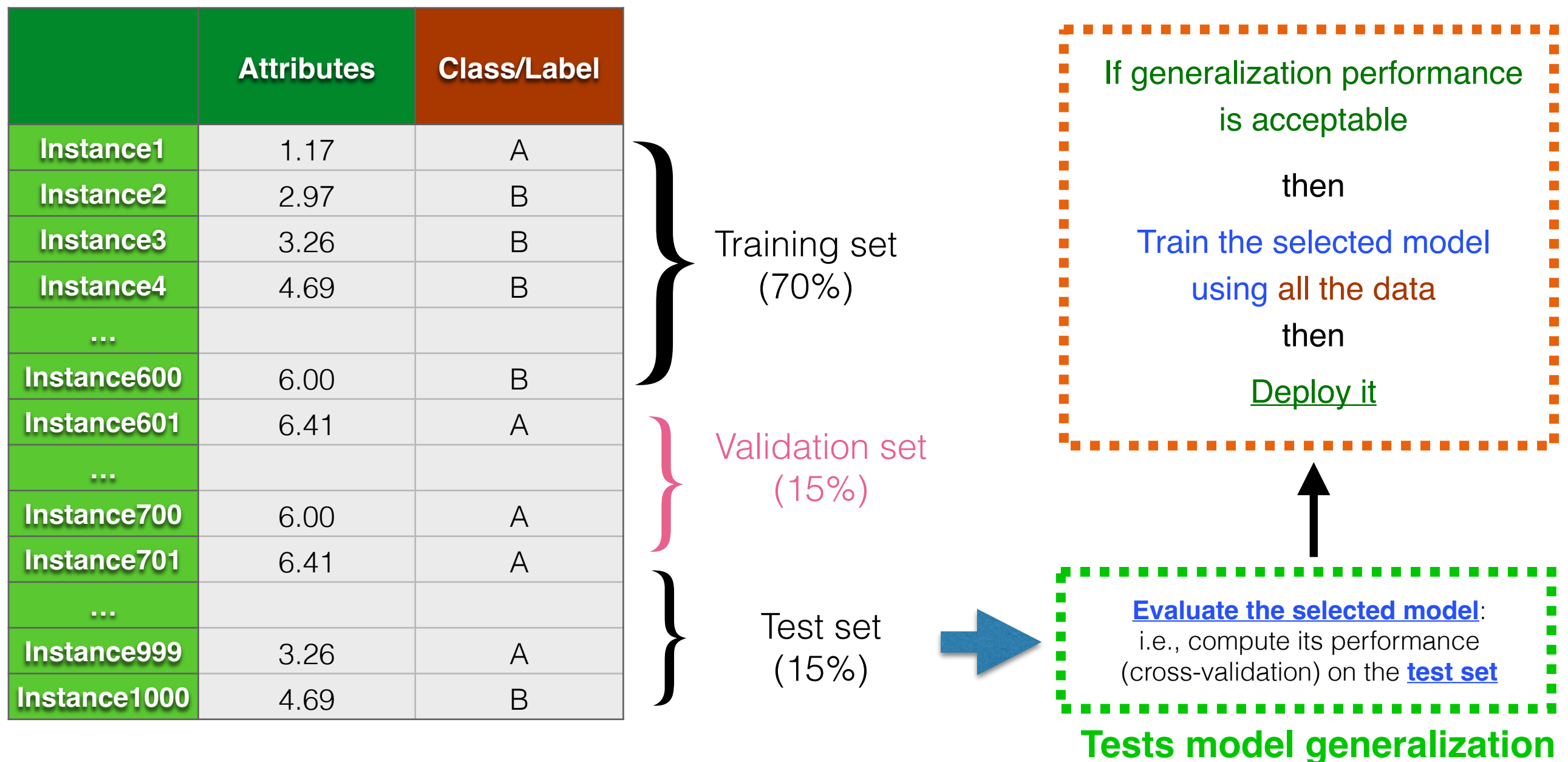
Model Selection

How do we compare learning algorithms (or their hyper-parameters) and identify the best?



Model Selection

How do we compare learning algorithms (or their hyper-parameters) and identify the best?



Model Selection

What if no models have acceptable performance?

1) Revisit your data

- Possible issues with dataset may cause model to perform poorly
- Revisit your data cleaning / preprocessing steps
- Check for missing / incorrect data, attribute normalization, etc.

2) Revisit your modeling choices

- Try out different algorithms
- Try using different sets of features / attributes

3) Revisit your hyperparameter tuning

- The hyperparameters you tested when identifying the best model may not be appropriate
- Try different values for the hyperparameters
- Explore different ranges for the hyperparameters

4) Get more data

- Model may be overfitting to the training and validation sets → that is, it will not generalize well to new data!
- Get more data!

If generalization performance
is acceptable

then

Train the selected model
using all the data

then

Deploy it



Machine Learning

CMPSCI 589

Bruno C. da Silva

bsilva@cs.umass.edu

Evaluation Metrics (2/2)

